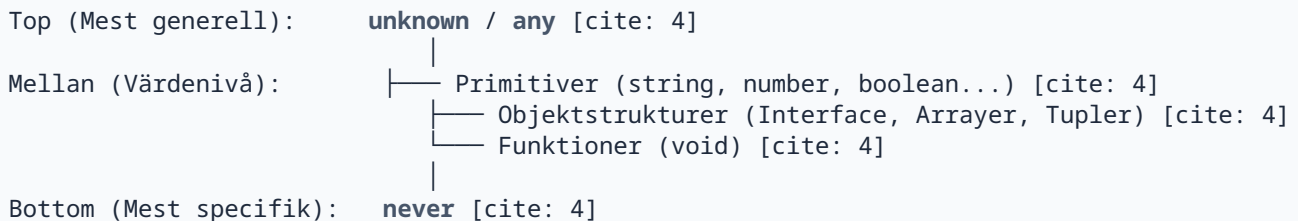


Typer i TypeScript

En visuell och strukturerad karta över TypeScript's typsystem

Man kan visualisera TypeScript's typsystem som ett **släktträd (en hierarki)**, där vissa typer är mer generella och andra är extremt specifika [cite: 4]. Nedan hittar du en fullständig karta över typerna i TypeScript, uppdelat i fem tydliga kategorier [cite: 4].

MENTAL KARTA: TYPSYSTEMETS HIERARKI



Kategoriserad översikt

1. De Primitiva typerna (De grundläggande)

Dessa kommer direkt från JavaScript och representerar enkla värden [cite: 4]:

- string** Text (t.ex. "Volvo") [cite: 4].
- number** Alla tal, inklusive decimaltal (t.ex. 42 eller 19.83) [cite: 4].
- boolean** Sant eller falskt (true/false) [cite: 4].
- null** Avsiktligt tomt värde [cite: 4].
- undefined** Odefinierat värde (variabeln har inte fått något värde än) [cite: 4].
- symbol** Unika identifierare (används sällan av nybörjare) [cite: 4].
- bigint** För extremt stora heltal [cite: 4].

2. Komplexa typer & Strukturer

Dese används för att gruppera och strukturera mer avancerad data [cite: 4]:

- Arrayer (type[])** Listor med data (t.ex. string[] eller number[]) [cite: 4].
- Objekt (object)** Representerar vilket icke-primitivt värde som helst [cite: 4].
- Interface & Types** Dina egna skräddarsydda modeller (t.ex. ICar eller IUser) [cite: 4].
- Tupler ([type, type])** En array med ett fast antal element där varje position har en bestämd typ, t.ex. [string, number] (perfekt för t.ex. koordinater [x, y]) [cite: 4].

3. Funktionstyper

void Används som returtyp för funktioner som inte returnerar något värde (de bara gör något) [cite: 4].

never Representerar värden som aldrig kan inträffa. Används till exempel för funktioner som alltid kastar ett felmeddelande eller hamnar i en oändlig loop och därmed aldrig "tar slut" [cite: 4].

4. Logiska typer (TypeScripts "superkrafter")

Dessa gör att du kan kombinera och skräddarsy typer på ett sätt som inte går i traditionella språk som C# [cite: 4]:

Union Types (|) Gör att en variabel kan vara en av flera typer (t.ex. string | number) [cite: 4].

Literal Types Du låser en variabel till ett eller flera exakta värden, inte bara en datatyp [cite: 4]:

```
let status: "success" | "error" | "loading"; [cite: 4]
status = "success"; // OK [cite: 4]
status = "pending"; // Fel! "pending" är inte tillåtet. [cite: 4]
```

Enums En numrerad lista med namngivna konstanter (precis som enum i C#) [cite: 4].

5. De "osäkra" nödutgångarna

any Stänger helt av TypeScript för den variabeln. Den kan vara vad som helst och ändras till vad som helst. (Bör undvikas!) [cite: 4].

unknown Den säkra kusinen till any. Den betyder också "vi vet inte vad detta är än" (t.ex. vid API-anrop), men TS tvingar dig att göra en kontroll (t.ex. if (typeof x === "string")) innan du får använda variabeln [cite: 4].